



LeetCode 709: To Lower Case

1. Problem Title & Link

Title: LeetCode 709: To Lower Case

Link: <https://leetcode.com/problems/to-lower-case/>

2. Problem Statement (Short Summary)

Given a string *s*, convert all uppercase English letters to lowercase and return the resulting string. Any lowercase letters, digits, or special characters should remain unchanged.

Example

Input:

s = "Hello"

Output:

"hello"

Explanation:

H → h

e → e

l → l

l → l

o → o

3. Examples (Input → Output)

Example 1

Input:

s = "Hello"

Output:

"hello"

Example 2

Input:

s = "here"

Output:

"here"

Explanation:

Already lowercase.

Example 3



Input:

s = "LOVELY"

Output:

"lovely"

Example 4

Input:

s = "Java123!"

Output:

"java123!"

Digits and symbols remain unchanged.

4. Constraints

$1 \leq s.length \leq 100$

s consists of printable ASCII characters.

Important:

Only uppercase English letters
need conversion.

5. Core Concept (Pattern / Topic)

String Manipulation

Secondary Topics:

- ASCII Values
- Character Processing

6. Thought Process (Step-by-Step Explanation)

Brute Force Thinking

Process every character.

For each character:

Is it uppercase?

If yes:

Convert to lowercase.

Otherwise:

Keep it unchanged.

ASCII Observation



Uppercase and lowercase letters have fixed ASCII values.

Character	ASCII
A	65
B	66
C	67
...	...
Z	90
a	97
b	98
c	99
...	...
z	122

Notice:

'a' - 'A' = 32

Similarly:

'b' - 'B' = 32

'c' - 'C' = 32

...

'z' - 'Z' = 32

Key Observation

To convert:

A → a

Simply add:

32

to its ASCII value.

Example:

A = 65

$65 + 32 = 97$

$97 = a$

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

"Hello"

Processing:



H → h

e → e

l → l

l → l

o → o

Result:

hello

ASCII Conversion

H

ASCII = 72

72 + 32

104

ASCII 104 = h

8. Pseudocode (Language Independent)

```
result = ""  
  
for each character c  
    if c between 'A' and 'Z'  
        convert using ASCII  
    append to result  
  
return result
```

9. Code Implementation

Python

Built-in Solution

```
class Solution:  
    def toLowerCase(self, s: str) -> str:
```



```
return s.lower()
```

Manual ASCII Solution

```
class Solution:
    def toLowerCase(self, s: str) -> str:

        result = []

        for ch in s:

            if 'A' <= ch <= 'Z':
                result.append(chr(ord(ch) + 32))
            else:
                result.append(ch)

        return ''.join(result)
```

Java

Built-in Solution

```
class Solution {

    public String toLowerCase(String s) {
        return s.toLowerCase();
    }

}
```

Manual ASCII Solution

```
class Solution {

    public String toLowerCase(String s) {

        StringBuilder result = new StringBuilder();

        for(char ch : s.toCharArray()) {

            if(ch >= 'A' && ch <= 'Z')
                result.append((char) (ch + 32));
            else
                result.append(ch);
        }

    }

}
```



```
        return result.toString();  
    }  
}
```

10. Time & Space Complexity

Built-in Solution

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Manual Solution

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Reason:

Every character is processed once.

11. Common Mistakes / Edge Cases

Mistake 1

Converting every character.

Wrong:

'1' + 32

Produces incorrect values.

Need:

Convert only A-Z.

Mistake 2

Using wrong ASCII offset.

Correct:

32

Not:

31

33

Edge Cases

Already Lowercase



Input:

"hello"

Output:

"hello"

Single Character

Input:

"A"

Output:

"a"

Digits

Input:

"123"

Output:

"123"

Special Characters

Input:

"@#\$%"

Output:

"@#\$%"

12. Detailed Dry Run (Step-by-Step Table)

Input:

s = "HeLLo"

Character	Uppercase?	Conversion	Result
H	Yes	h	h
e	No	e	he
L	Yes	l	hel
L	Yes	l	hell
o	No	o	hello

Final Output:

hello

13. Common Use Cases (Real-Life / Interview)

Case-Insensitive Search



Search:

APPLE

Apple

apple

All become:

apple

before comparison.

User Registration Systems

Store usernames in lowercase.

Example:

John

JOHN

john

treated as same user.

Database Queries

Normalize text before searching.

NLP (Natural Language Processing)

Text preprocessing often begins with:

Lowercase Conversion

14. Common Traps (Important!)

Trap 1

Forgetting ASCII relationship.

Lowercase = Uppercase + 32

Trap 2

Trying to convert digits.

Example:

5

should remain:

5

Trap 3

Using immutable strings inefficiently.



In Java:

```
result += ch;
```

inside loop is inefficient.

Use:

StringBuilder

instead.

15. Builds To (Related LeetCode Problems)

Progression:

LC 709 - To Lower Case



LC 125 - Valid Palindrome



LC 344 - Reverse String



LC 541 - Reverse String II



LC 151 - Reverse Words in a String

These build strong string manipulation skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Built-in Function	$O(n)$	$O(n)$	Good
ASCII Conversion	$O(n)$	$O(n)$	Better Learning
Bit Manipulation	$O(n)$	$O(n)$	Advanced

Approach 1: Built-in

Python:

```
s.lower()
```

Java:

```
s.toLowerCase()
```

Most practical.

Approach 2: ASCII

```
if 'A' <= c <= 'Z'
```

```
add 32
```



Shows understanding of character encoding.

Approach 3: Bit Manipulation (Advanced)

ASCII uppercase and lowercase differ by one bit.

Example:

A = 01000001

a = 01100001

Using bit operations can also convert cases.

Rarely asked but useful in low-level programming.

17. Why This Solution Works (Short Intuition)

Uppercase and lowercase letters differ by exactly **32** in ASCII.

Whenever an uppercase letter is found, adding 32 converts it to its lowercase equivalent while leaving all other characters unchanged.

18. Variations / Follow-Up Questions

Follow-Up 1

Convert lowercase to uppercase.

Example:

hello

Output:

HELLO

Follow-Up 2

Toggle case.

Example:

HeLLo

Output:

hElLo

Follow-Up 3

Count uppercase letters.

Example:

"JavaPY"

Output:

2

**Follow-Up 4**

Implement your own version of:

`toUpperCase()`

`toLowerCase()`

`swapCase()`

without using built-in functions.

Interview Takeaway

This problem is simple but introduces an important concept:

Character Encoding (ASCII)

Remember:

'A' = 65

'a' = 97

Difference = 32

Many string problems become easier once you understand how characters are represented internally.